Fontys

2024/
2025

Car Project

Design Document

VESELIN ATANASOV | ABDULLAH ADIL ÖNCÜ

Document History

V	*DATE*	*STATUS*	*AUTHOR*	*DESCRIPTION*	*UPDATE*
1.0	12/6/2024	Draft	Adil	Document Creation	Document Creation
1.1	12/13/2024	Draft	Veselin, Adil	Made the introduction, class diagrams, messaging protocol and the system overview and testing strategy	Introduction, Class Diagram(s), Message Protocol, System Overview
1.2	12/19/2024	Draft	Adil	Made the wiring diagrams and a few collaboration and sequence diagrams	System Overview, Collaboration Diagrams, Sequence Diagrams, Hardware Description

Table of Contents

Introduction

This document includes all the designs and specifications for the “Car simulation” project. The implementation of the project is done based on the designs shown in this document. All the designs will be changed and updated throughout the implementation process.

System Overview

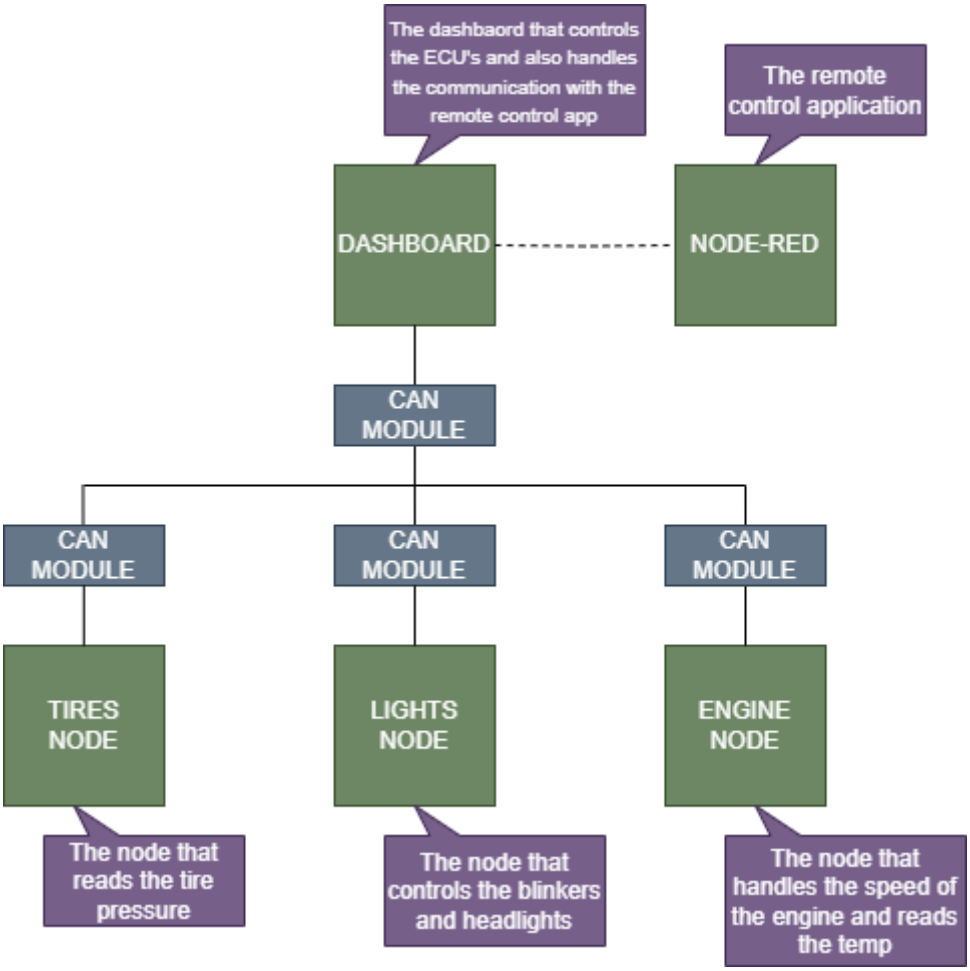

Figure 1: The System Overview of the project.

Above in the ***Figure 1*** the system overview diagram is being shown with comments for each of the node or ECU that is present in this project. All the nodes and ECU’s communicate through the CAN bus and the dashboard communicates with the node red remote application through Wi-Fi. In the below diagrams the further design of the project will be shown with various kinds of diagrams and descriptions.

Class Diagram(s)

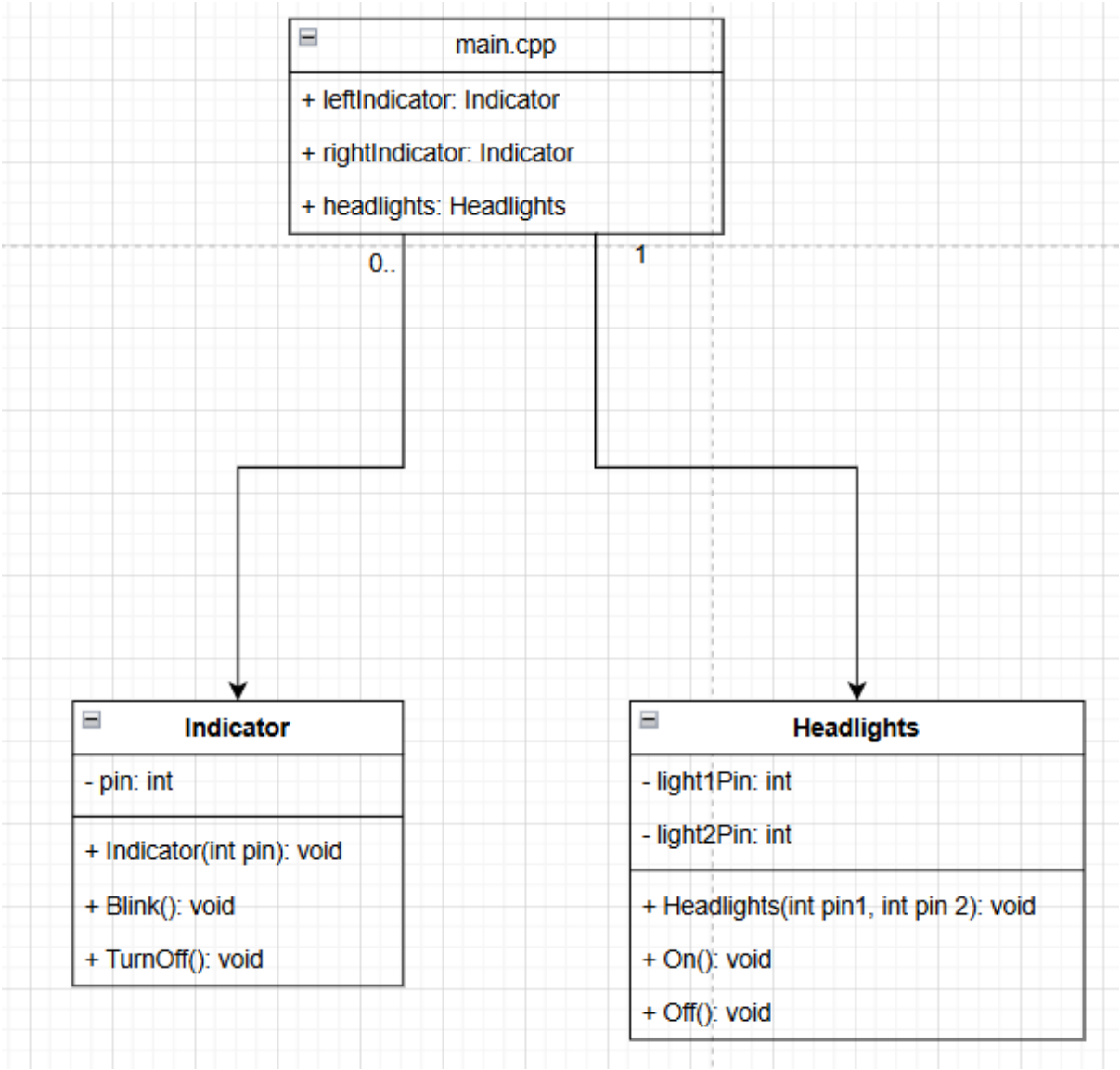

Figure 2: Lights ECU class diagram.

In ***Figure 2*** the Lights ECU class diagram shows two classes. The Indicator class and the headlights class. The Lights ECU has 2 Indicator objects for each indicator. This node is responsible for controlling the lights based on the messages from the dashboard and the app.

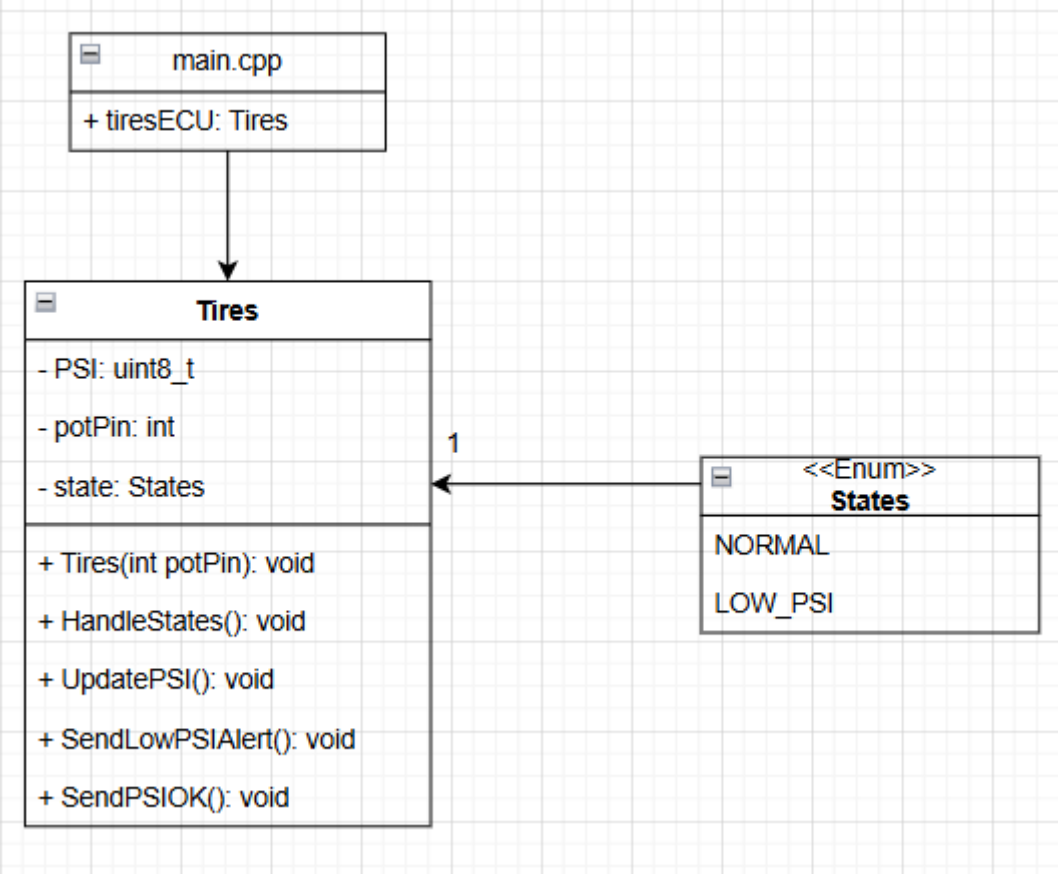

Figure 3: Tires ECU class diagram.

In ***Figure 3*** the class diagram of the tires node is shown. The tire class is responsible for updating the tire pressure every two seconds and sending it to the remote control app.

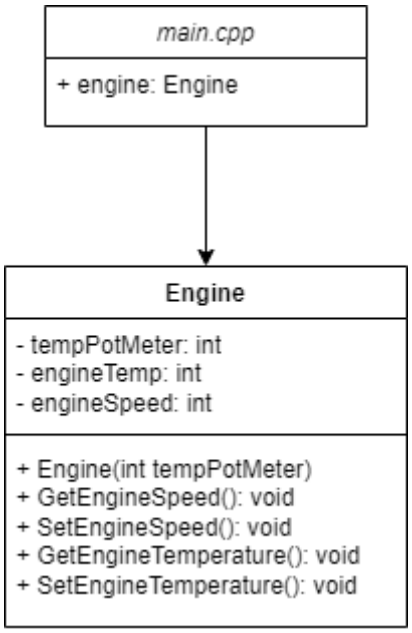

Figure 4: Engine ECU class diagram.

In ***Figure 4*** it can be seen the engine will have a potentiometer that will control the simulate the engines temperature. And 2 variables to keep track of the engine temperature and engine speed.

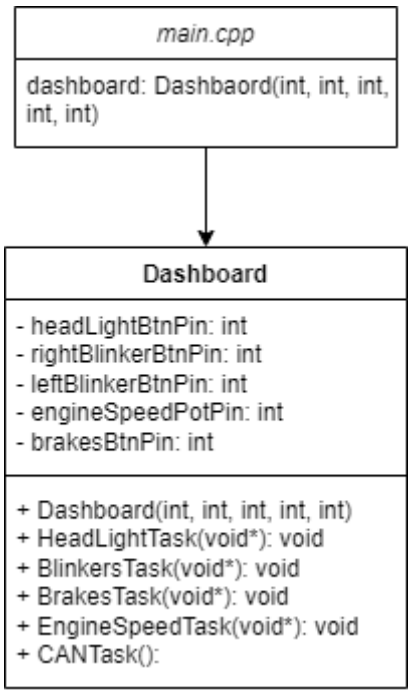

Figure 5: Dashboard ECU class diagram.

In ***Figure 5*** it can be seen that the Dashboard ECU will have 4 buttons and a potentiometer. The 4 buttons will be there to control the headlights, the right and left blinkers and the brake. The potentiometer will simulate a gas pedal and control the speed of the engine. For each ECU that the messages are going to be sent a task will be made for it.

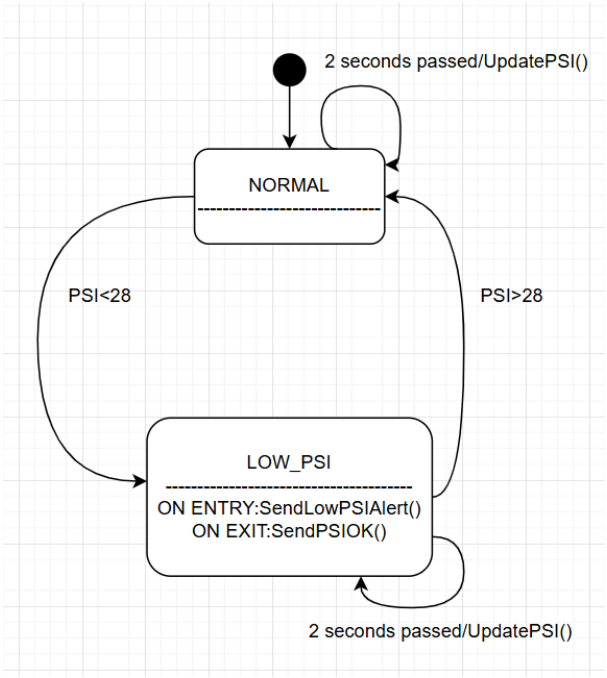

Figure 6: Tires ECU state diagram.

Figure 6 shows the state diagram of the tires node. It shows that in state "NORMAL" the PSI is updated every two seconds and the program transitions to LOW_PSI if the tire pressure is lower than 28 psi. When it enters the state the SendLowPSIAlert() function is executed and then the pressure is still updated every two seconds. If the pressure restores to normal values the SendPSIOK() function is executed and the state changes to "NORMAL".

Collaboration Diagram(s)

Here are some Collaboration diagrams to understand the order of actions being taken by the nodes and the system itself:

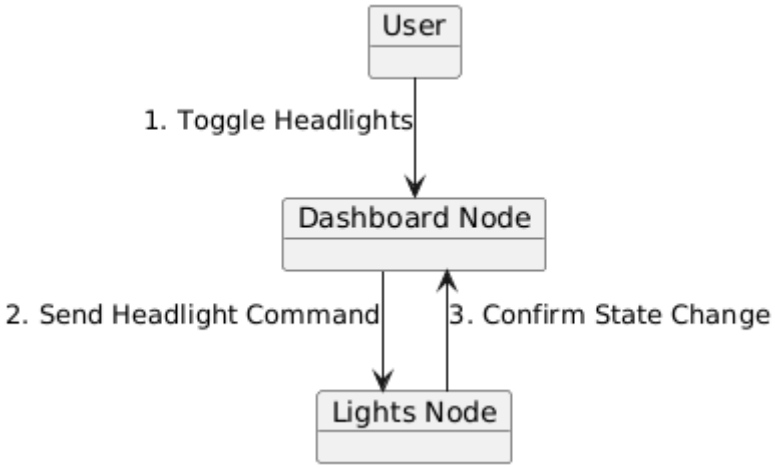

Figure 7: Collaboration diagram of toggling headlights.

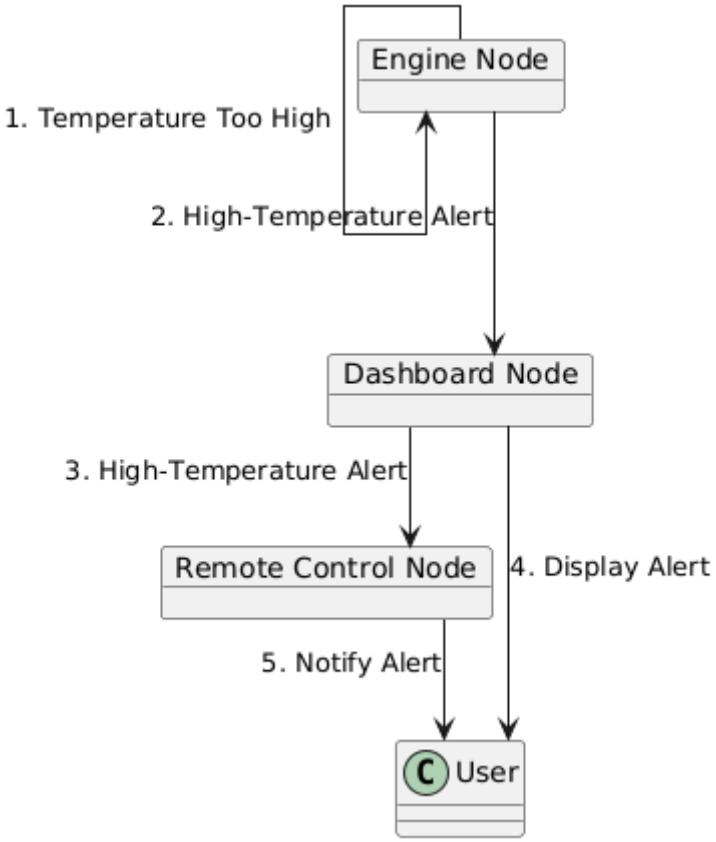

Figure 8: Collaboration diagram of sending high temperature alert.

Sequence Diagram(s)

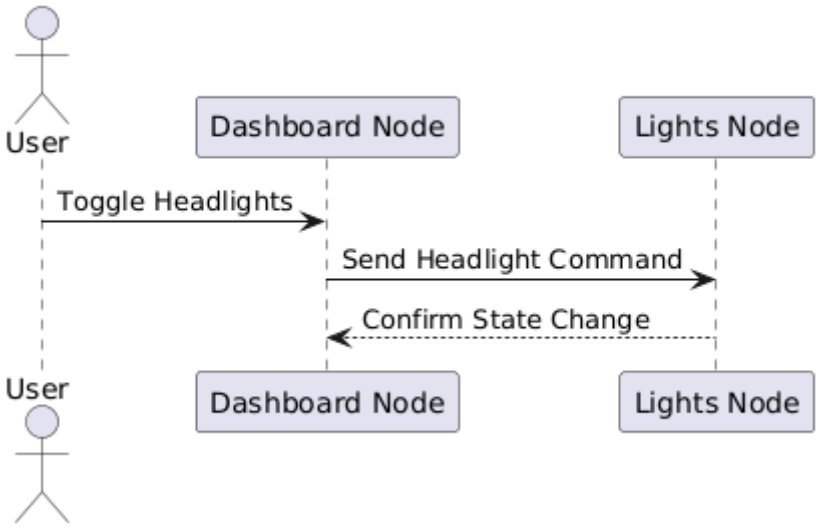

Figure 9: Sequence diagram of toggling headlights.

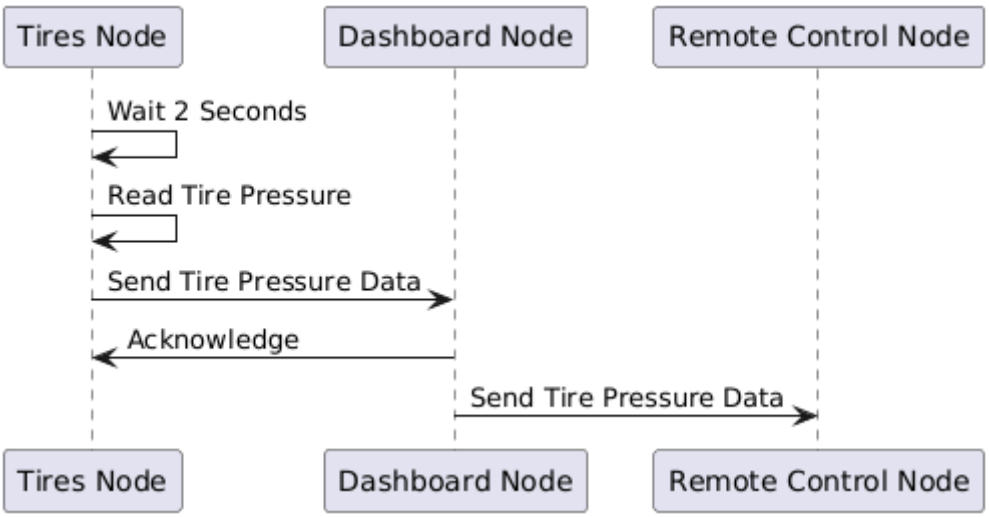

Figure 10: Sequence diagram of reading and sending tire pressure data.

There has not been made a collaboration and a sequence diagram for each functionality of the system sue to most of the sequences and collaborations being similar to each other. For example, the reading of tire pressure and the reading of engine temperature are basically the same with only names different. Same goes for indicators and headlights.

Hardware Description

NAME	AMOUNT	COMPONENT
Btn	4	Push Button
LED	4	LED: Two Pin (yellow)
LCD	1	LCD Display (16 pin)
I2C Module	1	LCM1632 IIC Module
Capacitor	4	Electrolytic Capacitor (4.7 µF)
R10000	4	Resistor (10 kΩ)
R330	4	Resistor (330 Ω)
Pot	3	Potentiometer
ESP	4	ESP32-CP2102-MICRO

Table 1: Tabel of the used components for this project

It can be clearly seen which components have been used in this project in ***Table 1***. The 4 buttons will be on the Dashboard ECU and what they will be for is explained clearly in ***Class Diagram(s)***. Same goes for the LEDs. The LCD and the I2C Module will be for displaying information on the Dashboard ECU. The information that it will display are the state of the nodes and warnings. The capacitors and the resistors will be used for the Buttons and the LEDs. The Potentiometers will be used to simulate engine temperature, engine speed and tire pressure.

Wiring Diagrams

Here are the wiring diagrams of the all nodes in the system to get a better understanding of how it will function and look like:

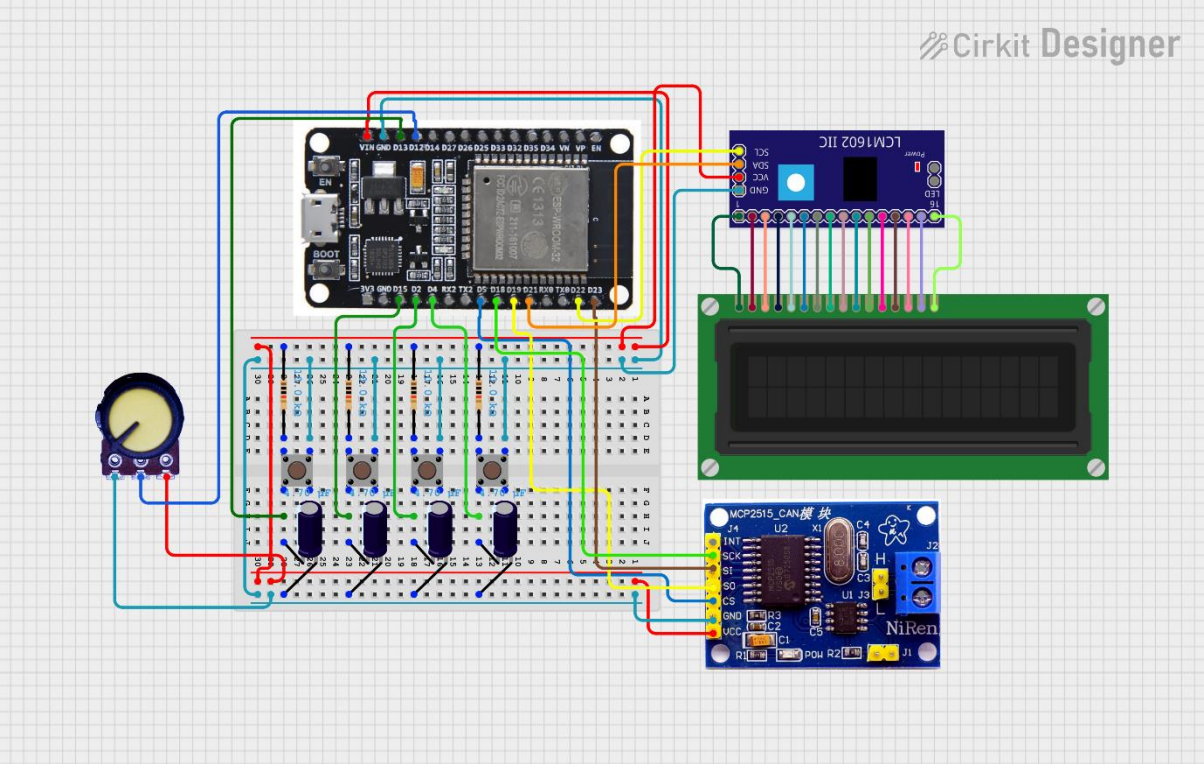

Figure 11: Wiring diagram of the Dashboard Node.

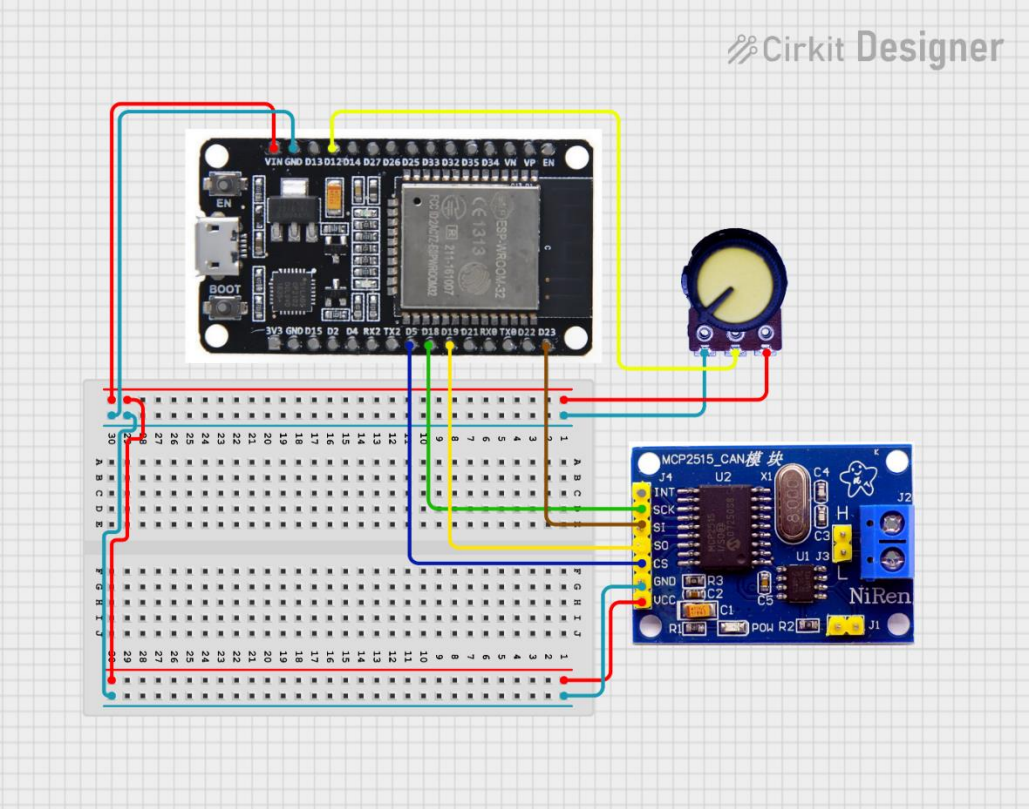

Figure 12: Wiring diagram of the Engine Node.

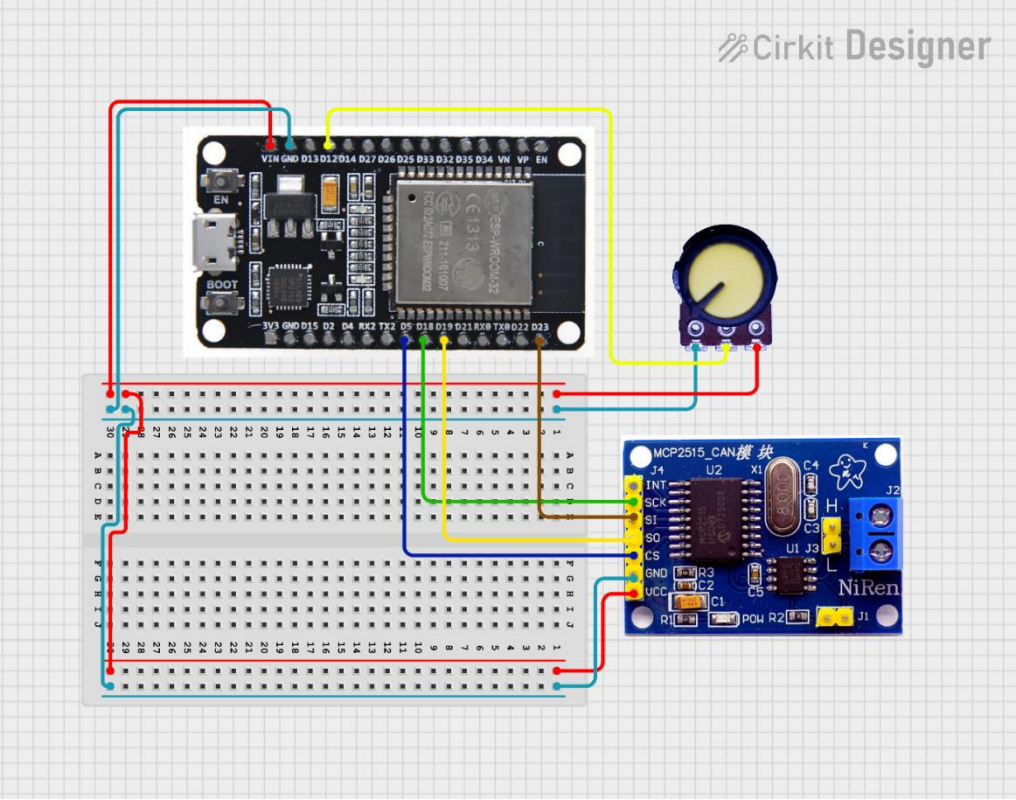

Figure 13: Wiring diagram of the Tires Node.

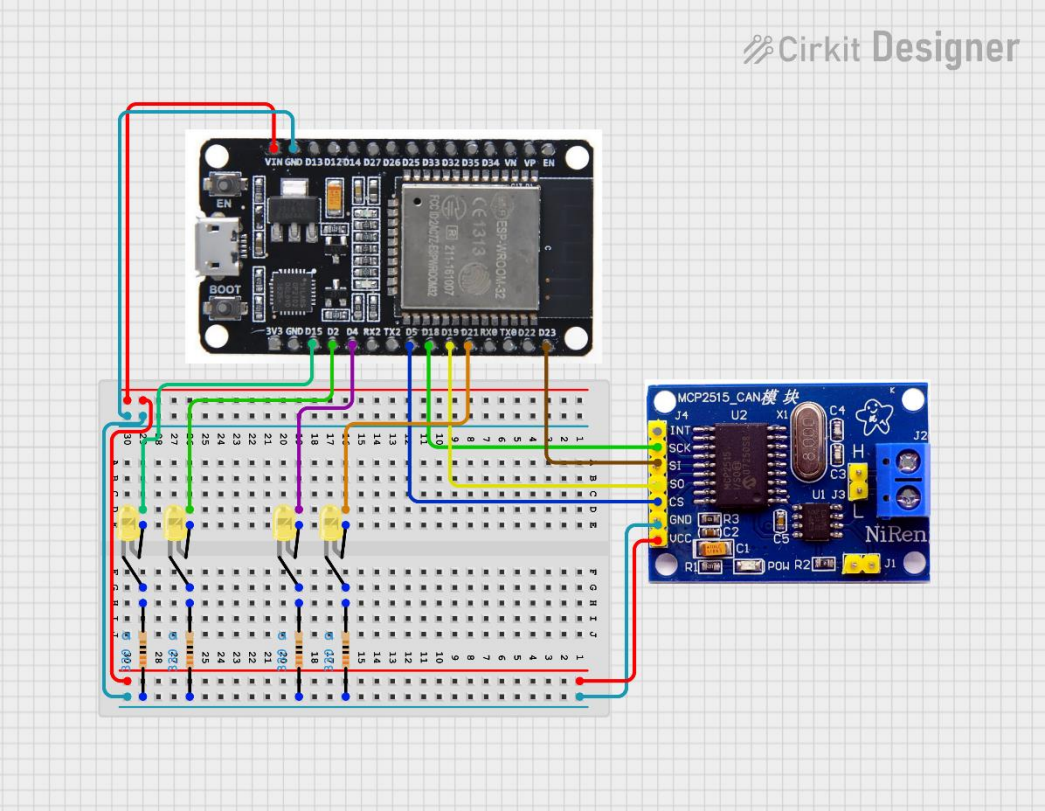

Figure 14: Wiring diagram of the Lights Node.

Message Protocol

An important aspect of CAN, is arbitration. In the arbitration phase, it is decided which message will be put on the bus. The lower the ID of the message, the higher the priority.

Message description	Message priority	Message ID	Message Data
Whenever the brakes are pressed, a message to the engine is sent and the engine stops.	High	0x01	Byte 1:0x01
Control of the speed of the engine from dashboard.	High	0x02	Byte 1:SpeedLow Byte 2:SpeedHigh
High temperature alert from engine to dashboard	Medium	0x03	Byte 1:H Byte 2:I Byte 3:G Byte 4:H
Low pressure alert from tires to dashboard.	Medium	0x04	Byte 1:L Byte 2:O Byte 3:W

Controlling blinkers from dashboard to lights.	Low	0x05	Byte 1:L/R (depending on the blinker) Byte 2:1/0 (depending on state of blinker)
Controlling headlights from dashboard to lights.	Low	0x06	Byte 1:1/0(depending on state of headlights)

Table 2: The Message Protocol for the project.

Testing Strategy

Testing is going to be executed by connecting nodes and observing system behaviour. The system will be deemed completed.

Phase 1: Talking Skeleton

In the first phase, all the nodes will be implemented with basic functionality for sending and receiving messages. The test is to send messagess wich are defined in the message protcol design and print the received messages via the serial monitor.

Phase 2: Individual component testing

In this phase all the happy flow functionality of the individual components will be tested. The Tires class will be tested using mock test to test state transitioning and behaviour. The other components will be tested in collaboration with the Dashboard node. They will have to sent the commands and execute them as expected and also sent messages according to their functionality and design.

Light node-Test is successful when it receives command to turn headlights on/off and the headlight leds act accordingly and when it receives command to turn individual blinkers on and off and the blinker leds act accordingly.

Tires node-Test is succesfull when the potentiometer value(PSI) goes under the minimum normal value and an alert is sent to the dashboard and application(which should receive pressure readings every 2 seconds) and then when the PSI value goes back to normal value, the message to cancel the alert on the dashboard and app is sent to and acknowledged by the stated receivers.

Engine node-Test is successful when the speed of the engine is properly updated by the application(application also receives the engine speed every 2 seconds) and the

dashboard. When the potentiometer value(engine temperature) goes above normal range, an alert is sent to the dashboard and the app.

Dashboard node-Test is successful when it successfully controls, engine speed, brakes and lights(headlights, indicators), and also shows the high engine temperature and low tire pressure alerts.

Phase 3:Full integration testing

The final phase should test the behaviour of the system with all components present. All the functionality mentioned above in Phase 2, should still be present with all nodes connecting and interacting with the dashboard and the remote control app.